

ASSIGNMENT:-07

Name: Rajdeep Das

RollNo: CSB25203

Subject: Advanced Programming

```
from __future__ import annotations
```

```
import math
```

```
import time
```

```
import tracemalloc
```

```
import random
```

```
from collections import defaultdict
```

```
from functools import reduce
```

```
from typing import Callable
```

```
import heapq
```

```
SAMPLE_LOGS: list[dict] = [
```

```
    {"user": "S101", "action": "YouTube", "duration": 45.0},
```

```
    {"user": "S102", "action": "Instagram", "duration": 30.0},
```

```
    {"user": "S101", "action": "Zoom", "duration": 60.0},
```

```
    {"user": "S103", "action": "YouTube", "duration": 20.0},
```

```
    {"user": "S102", "action": "Zoom", "duration": 50.0},
```

```
    {"user": "S104", "action": "Netflix", "duration": 90.0},
```

```
    {"user": "S101", "action": "Instagram", "duration": 25.0},
```

```
    {"user": "S103", "action": "Zoom", "duration": 40.0},
```

```
    {"user": "S104", "action": "YouTube", "duration": 35.0},
```

```
    {"user": "S105", "action": "Discord", "duration": 55.0},
```

```

{"user": "S102", "action": "Netflix", "duration": 70.0},
{"user": "S105", "action": "YouTube", "duration": 80.0},
{"user": "S103", "action": "Discord", "duration": 15.0},
{"user": "S104", "action": "Zoom", "duration": 45.0},
{"user": "S101", "action": "Discord", "duration": 10.0},
]

```

```

deftotal_time_per_user(logs: list[dict]) -> dict[str, float]:

```

```

    def _acc(acc: defaultdict, entry: dict) -> defaultdict:

```

```

        acc[entry["user"]] += entry["duration"]

```

```

        return acc

```

```

    return dict(reduce(_acc, logs, defaultdict(float)))

```

```

defmost_active_users(logs: list[dict], k: int) -> list[str]:

```

```

    totals = total_time_per_user(logs)

```

```

    return [u for u, _ in heapq.nlargest(k, totals.items(), key=lambda x: x[1])]

```

```

def unique_actions(logs: list[dict]) -> set[str]:

```

```

    return {entry["action"] for entry in logs}

```

```

ACTIONS = ["YouTube", "Instagram", "Zoom", "Netflix", "Discord",
           "Twitter", "Reddit", "Spotify", "WhatsApp", "Gmail"]

```

```

defgenerate_logs(n: int) -> list[dict]:

```

```

    num_users = max(2, int(math.sqrt(n)))

```

```

    users = [f"S{str(i).zfill(4)}" for i in range(num_users)]

```

```

    return [

```

```

{
    "user": random.choice(users),
    "action": random.choice(ACTIONS),
    "duration": round(random.uniform(5.0, 120.0), 2),
}
for_inrange(n)
]

```

defmeasure_time(fn: Callable, *args, repeats: int = 7) -> float:

```

times = []
for_inrange(repeats):
    start=time.perf_counter()
    fn(*args)
    times.append(time.perf_counter() - start)
times.sort()
returntimes[len(times) // 2]

```

defmeasure_space_bytes(fn: Callable, *args) -> int:

```

tracemalloc.start()
fn(*args)
_, peak = tracemalloc.get_traced_memory()
tracemalloc.stop()
return peak

```

def fit_log_log(xs: list[int], ys: list[float]) -> tuple[str, float]:

```

lx = [math.log(x) for x in xs]
ly = [math.log(max(y, 1e-15)) for y in ys]

```

```
n = len(lx) mx = sum(lx)/n my = sum(ly) /n slope=  
(sum((x-mx)*(y-my)for x,y in zip(lx, ly)) /
```

```
sum((x - mx) ** 2 for x in lx))
```

```
if slope < 0.20: label = "O(1)"  
elif slope < 0.70: label = "O( $\sqrt{n}$ )"  
elif slope < 1.05: label = "O(n)"  
elif slope < 1.60: label = "O(n log n)"  
elif slope < 2.20: label = "O( $n^2$ )"  
else:      label = f"O( $n^{\text{slope:.2f}}$ )"
```

```
return label, round(slope, 4)
```

```
def benchmark(  
    fn: Callable,  
    sizes: list[int],  
    args_builder:Callable[[list[dict]],tuple],  
    label: str,  
) -> dict:  
    times_s: list[float] = []  
    peaks_b: list[int] =[]  
  
    for n in sizes:  
        logs = generate_logs(n)  
        args = args_builder(logs)  
        times_s.append(measure_time(fn,*args))  
        peaks_b.append(measure_space_bytes(fn, *args))
```

```
time_label, time_slope = fit_log_log(sizes, times_s)
space_label, space_slope = fit_log_log(sizes, [max(p, 1) for p in peaks_b])
```

```
return {
    "label": label,
    "sizes": sizes,
    "times_us": [round(t * 1e6, 2) for t in times_s],
    "spaces_kb": [round(p / 1024, 2) for p in peaks_b],
    "time_label": time_label,
    "time_slope": time_slope,
    "space_label": space_label,
    "space_slope": space_slope,
}
```

```
def_bar(value: float, max_val: float, width: int = 18) -> str:
    filled = int(round(value / max_val * width)) if max_val else 0
    return "█" * filled + "\\\\" * (width - filled)
```

```
def _section(title: str) -> None:
    print(f"\n{'—' * 64}")
    print(f" {title}")
    print(f"{'—' * 64}")
```

```
def print_report(logs: list[dict], k: int = 3) -> None:
    _section("ACTIVITY LOG ANALYZER — RESULTS")

    totals = total_time_per_user(logs)
    max_t = max(totals.values())
```

```

print(f"\n{'User':<8}{Total (min)':>12} Bar")
print(f"{'-'*8}{-'*12}{-'*20}")
for user, mins in sorted(totals.items(), key=lambda x: -x[1]):
    print(f"{'user':<8}{mins:>12.1f}{_bar(mins, max_t)}")

_section(f"Top-{k} Most Active Users")
for rank, user in enumerate(most_active_users(logs, k), 1):
    print(f"#{rank}{user} ({totals[user]:.1f} min)")

_section("Unique Actions")
acts = unique_actions(logs)
print(f"Count: {len(acts)}")
print(f"Set: {sorted(acts)}")

_section("EMPIRICAL COMPLEXITY ANALYSIS (actually computed, not assumed)")
print("Each function is run at 6 input sizes.")
print("log(time) vs log(n) is fitted via OLS regression.")
print("The slope determines the Big-O class.\n")

sizes = [500, 1_000, 2_000, 5_000, 10_000, 20_000]

results = [
    benchmark(total_time_per_user, sizes,
               args_builder=lambda logs: (logs,),
               label="total_time_per_user"),
    benchmark(most_active_users, sizes,
               args_builder=lambda logs: (logs, 3),
               label="most_active_users (k=3)"),
    benchmark(unique_actions, sizes,
               args_builder=lambda logs: (logs,),
               label="unique_actions"),

```

]

forin results:

```
print(f" └─ {r['label']}")
print(f" | log-log slope (time) = {r['time_slope']:+.4f} → {r['time_label']}")
print(f" | log-log slope (space) = {r['space_slope']:+.4f} → {r['space_label']}")
print(f" |")
print(f" | {'n':>7}{Time (μs)':>10}{Mem (KB)':>9} Time growth bar")
print(f" | {'-'*7}{-'*10}{-'*9}{-'*20}")
max_time = max(r['times_us'])
for n, t, s in zip(r['sizes'], r['times_us'], r['spaces_kb']):
    print(f" | {n:>7}{t:>10.2f}{s:>9.2f}{_bar(t, max_time)}")
print(f" └─{'-'*57}\n")
```

_section("COMPLEXITY SUMMARY (empirically derived via log-log regression)")

```
print(f"\n {'Function':<30}{s['slope':>7]}{'Time':^12}{s['Space':^12}]")
print(f" {'-'*30}{-'*7}{-'*12}{-'*12}"]
```

forin results:

```
print(f" {r['label']:<30}{r['time_slope']:>+7.4f} "
      f"{r['time_label']:^12}{r['space_label']:^12}")
print()
print(" How the slope maps to Big-O:")
print(" slope < 0.20 → O(1)")
print(" 0.20 – 0.70 → O(√n)")
print(" 0.70 – 1.05 → O(n)")
print(" 1.05 – 1.60 → O(n log n)")
print(" 1.60 – 2.20 → O(n²)")
print()
```

if __name__ == "__main__":

```
    random.seed(42)
    print_report(SAMPLE_LOGS, k=3)
```